

Program - 3

Run a two-broker Apache Kafka cluster on a single machine using Docker Desktop, then create a topic, produce, and consume messages with simple Java applications. Everything is step-by-step.

step 1 -cd kafka

step 2 - mkdir kafka-multi-broker

step 3 - cd kafka-multi-broker/

step 4 - nano docker-compose.yml

copy the below code and give a right click from mouse & paste inside it.

version: '3.8'

networks:

kafka-net:
driver: bridge

services:

zookeeper:

image: wurstmeister/zookeeper

container_name: zookeeper

ports:

- "2181:2181"

environment:

ZOOKEEPER_CLIENT_PORT: 2181

ZOOKEEPER_TICK_TIME: 2000

networks:

- kafka-net

kafka1:

image: wurstmeister/kafka

container_name: kafka1

ports:

- "9092:9092"

- "29092:29092"

environment:

KAFKA_BROKER_ID: 1

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:

INSIDE:PLAINTEXT,OUTSIDE:PLAINTEXT

KAFKA_ADVERTISED_LISTENERS: INSIDE://kafka1:9092,OUTSIDE://localhost:29092

KAFKA_LISTENERS: INSIDE://0.0.0.0:9092,OUTSIDE://0.0.0.0:29092

KAFKA_INTER_BROKER_LISTENER_NAME: INSIDE

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 2

KAFKA_MIN_INSYNC_REPLICAS: 1

volumes:

- /var/run/docker.sock:/var/run/docker.sock

depends_on:

- zookeeper

networks:

- kafka-net

kafka2:

image: wurstmeister/kafka:2.13-2.8.1

container_name: kafka2

ports:

- "9093:9093"
- "29093:29093"

environment:

KAFKA_BROKER_ID: 2

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:

INSIDE:PLAINTEXT,OUTSIDE:PLAINTEXT

KAFKA_ADVERTISED_LISTENERS: INSIDE://kafka2:9093,OUTSIDE://localhost:29093

KAFKA_LISTENERS: INSIDE://0.0.0.0:9093,OUTSIDE://0.0.0.0:29093

KAFKA_INTER_BROKER_LISTENER_NAME: INSIDE

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 2

KAFKA_MIN_INSYNC_REPLICAS: 1

volumes:

- /var/run/docker.sock:/var/run/docker.sock

depends_on:

- zookeeper

networks:

- kafka-net

kafka-ui:

image: provectuslabs/kafka-ui:latest

container_name: kafka-ui

ports:

- "8080:8080"

environment:

KAFKA_CLUSTERS_0_NAME: local

KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka1:9092,kafka2:9093

KAFKA_CLUSTERS_0_ZOOKEEPER: zookeeper:2181

depends_on:

- kafka1
- kafka2

networks:

- kafka-net

step 5 - press ctrl+X then ctrl+Y afterwards press enter

step 6 - cat docker-compose.yml

step 7 - docker compose up -d

step 8 - open a new terminal & navigate to kafka-multi-broker

step 9 - cd kafka

step 10 - cd kafka-multi-broker/

step 11 - mkdir -p java-app/src/main/java/com/example

step 12 - cd java-app

step 13 - nano pom.xml

copy the below code and give a right click from mouse & paste inside it.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>kafka-java-app</artifactId>
  <version>1.0.0</version>
  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
      <version>3.7.0</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
          <cleanupDaemonThreads>>false</cleanupDaemonThreads>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

step 14 - press ctrl+X then ctrl+Y afterwards press enter

step 15 - cat pom.xml

step 16 - nano src/main/java/com/example/ProducerApp.java

copy the below code and give a right click from mouse & paste inside it.

```

package com.example;

import org.apache.kafka.clients.producer.*;
import java.util.Properties;

public class ProducerApp {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:29092,localhost:29093");
        props.put(ProducerConfig.ACKS_CONFIG, "all");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
"org.apache.kafka.common.serialization.StringSerializer");
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
"org.apache.kafka.common.serialization.StringSerializer");

        try (KafkaProducer<String, String> producer = new KafkaProducer<>(props)) {
            for (int i = 0; i < 10; i++) {
                String key = Integer.toString(i);
                String value = "This is message " + i;
                ProducerRecord<String, String> record = new ProducerRecord<>("test", key, value);
                producer.send(record, (metadata, exception) -> {
                    if (exception == null) {
                        System.out.printf("Produced -> topic=%s partition=%d offset=%d key=%s value=
%s%n",
                                metadata.topic(), metadata.partition(), metadata.offset(), key, value);
                    } else {
                        exception.printStackTrace();
                    }
                });
            }
        }
    }
}

```

step 17 - press ctrl+X then ctrl+Y afterwards press enter

step 18 - nano src/main/java/com/example/ConsumerApp.java

copy the below code and give a right click from mouse & paste inside it.

```

package com.example;

import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class ConsumerApp {
    public static void main(String[] args) {
        Properties props = new Properties();

```

```

        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:29092,localhost:29093");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "my-group");
        props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, "true");
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());

        try (KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props)) {
            consumer.subscribe(Collections.singletonList("test"));
            System.out.println("Consuming messages... (Ctrl+C to exit)");
            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(1000));
                for (ConsumerRecord<String, String> r : records) {
                    System.out.printf("Consumed <- partition=%d offset=%d key=%s value=%s%n",
                        r.partition(), r.offset(), r.key(), r.value());
                }
            }
        }
    }
}

```

step 19 - nano src/main/java/com/example/CreateTopic.java

copy the below code and give a right click from mouse & paste inside it.

```

package com.example;

import org.apache.kafka.clients.admin.*;
import java.util.*;

public class CreateTopic {
    private static final List<String> BOOTSTRAP = Arrays.asList("localhost:29092",
"localhost:29093");
    private static final String TOPIC = "test";
    private static final int PARTITIONS = 2;
    private static final short REPLICATION = 2;

    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, String.join(",",
BOOTSTRAP));
        try (AdminClient admin = AdminClient.create(props)) {
            NewTopic newTopic = new NewTopic(TOPIC, PARTITIONS, REPLICATION);
            try {
                admin.createTopics(Collections.singletonList(newTopic)).all().get();
                System.out.printf("Created topic '%s' with %d partitions, RF=%d%n", TOPIC,
PARTITIONS, REPLICATION);
            } catch (Exception e) {

```

```

        System.out.println("Topic may already exist: " + e.getMessage());
    }
}
}
}
}

```

step 20 - ctrl+X then ctrl+Y afterwards enter

step 21 - execute these 2 commands at a time in terminal

```

mvn clean package
mvn exec:java -Dexec.mainClass=com.example.CreateTopic

```

step 22 - mvn exec:java -Dexec.mainClass=com.example.ProducerApp

step 23 - open a new terminal and type

```

mvn exec:java -Dexec.mainClass=com.example.ConsumerApp

```

docker compose.yml = new

services:

broker-1:

image: confluentinc/cp-kafka:7.6.0

hostname: broker-1

container_name: broker-1

ports:

- "9092:9092"

environment:

KAFKA_NODE_ID: 1

KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:

'CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT'

KAFKA_ADVERTISED_LISTENERS: 'PLAINTEXT://broker-

1:29092,PLAINTEXT_HOST://localhost:9092'

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 2

KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0

KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1

KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 2

KAFKA_PROCESS_ROLES: 'broker,controller'

KAFKA_CONTROLLER_QUORUM_VOTERS: '1@broker-1:29093,2@broker-2:29093'

KAFKA_LISTENERS:

'PLAINTEXT://0.0.0.0:29092,CONTROLLER://0.0.0.0:29093,PLAINTEXT_HOST://0.0.0.0:9092'

KAFKA_INTER_BROKER_LISTENER_NAME: 'PLAINTEXT'

KAFKA_CONTROLLER_LISTENER_NAMES: 'CONTROLLER'

KAFKA_LOG_DIRS: '/tmp/kraft-combined-logs'